

Practical – Week6

Session 6:

1. Create a client-server application using Named Pipes (FIFOs). The client sends messages and the server processes and responds to them. Analyze FIFO behavior when multiple clients communicate simultaneously.

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <fcntl.h>
#include <sys/stat.h>
#define FIFO1 "client_to_server"
#define FIFO2 "server_to_client"
int main() {
    char message[100];
    int fd_read, fd_write;
    mkfifo(FIFO1, 0666);
    mkfifo(FIFO2, 0666);
    printf("Server started...\n");
    while (1) {
        fd_read = open(FIFO1, O_RDONLY);
        read(fd_read, message, sizeof(message));
        close(fd_read);
        printf("Client: %s\n", message);
        // Exit request
        if (strcmp(message, "exit") == 0) {
            break;
        }
        // Process message
        char response[150];
        snprintf(response, sizeof(response),
                 "Server processed: %s", message);
        // Open FIFO for writing
        fd_write = open(FIFO2, O_WRONLY);
        write(fd_write, response, strlen(response) + 1);
        close(fd_write);
    }
    unlink(FIFO1);
    unlink(FIFO2);
    printf("Server stopped.\n");
    return 0;
}
```

```

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <fcntl.h>
#define FIFO1 "client_to_server"
#define FIFO2 "server_to_client"
int main() {
    char message[100];
    char response[150];
    int fd_write, fd_read;
    printf("Enter message: ");
    fgets(message, sizeof(message), stdin);
    message[strcspn(message, "\n")] = '\0';
    // Open FIFO for writing
    fd_write = open(FIFO1, O_WRONLY);
    write(fd_write, message, strlen(message) + 1);
    close(fd_write);
    // Exit request
    if (strcmp(message, "exit") == 0) {
        return 0;
    }
    // Open FIFO for reading
    fd_read = open(FIFO2, O_RDONLY);
    read(fd_read, response, sizeof(response));
    close(fd_read);
    printf("Server response: %s\n", response);
    return 0;
}

```

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <fcntl.h>
#define FIF01 "client_to_server"
#define FIF02 "server_to_client"
int main(int argc, char *argv[]) {

    if (argc != 2) {
        printf("Usage: %s message\n", argv[0]);
        return 1;
    }

    char response[150];

    int fd_write, fd_read;

    // Send message to server
    fd_write = open(FIF01, O_WRONLY);

    write(fd_write, argv[1], strlen(argv[1]) + 1);

    close(fd_write);

    // Receive response
    fd_read = open(FIF02, O_RDONLY);

    read(fd_read, response, sizeof(response));

    close(fd_read);

    printf("Client received: %s\n", response);

    return 0;
}
```

```
kavya_aluru@Pandu: ~/OS_Lab x kavya_aluru@Pandu: ~/OS_La x + v - □ X
kavya_aluru@Pandu:~$ cd ~/OS_Lab
kavya_aluru@Pandu:~/OS_Lab$ ./fifo_client
Enter message: Hello derver
Server response: Server processed: Hello derver
kavya_aluru@Pandu:~/OS_Lab$ ./fifo_client
Enter message: How are you?
Server response: Server processed: How are you?
kavya_aluru@Pandu:~/OS_Lab$ ./fifo_client_auto "Client1 Hello"
Client received: Server processed: Client1 Hello
kavya_aluru@Pandu:~/OS_Lab$ ./fifo_client_auto "Client2 Hello"
Client received: Server processed: Client2 Hello
kavya_aluru@Pandu:~/OS_Lab$ ls -l client_to_server server_to_client
prw-r--r-- 1 kavya_aluru kavya_aluru 0 Sep 16 06:44 client_to_server
prw-r--r-- 1 kavya_aluru kavya_aluru 0 Sep 16 06:44 server_to_client
kavya_aluru@Pandu:~/OS_Lab$ file client_to_server
client_to_server: fifo (named pipe)
kavya_aluru@Pandu:~/OS_Lab$ ./fifo_client
Enter message: exit
kavya_aluru@Pandu:~/OS_Lab$ |

kavya_aluru@Pandu: ~/OS_La x kavya_aluru@Pandu: ~/OS_Lat x + v - □ X
kavya_aluru@Pandu:~$ cd ~/OS_Lab
kavya_aluru@Pandu:~/OS_Lab$ nano fifo_server.c
kavya_aluru@Pandu:~/OS_Lab$ nano fifo_client.c
kavya_aluru@Pandu:~/OS_Lab$ nano fifo_client_auto.c
kavya_aluru@Pandu:~/OS_Lab$ gcc fifo_server.c -o fifo_server
kavya_aluru@Pandu:~/OS_Lab$ gcc fifo_client.c -o fifo_client
kavya_aluru@Pandu:~/OS_Lab$ gcc fifo_client_auto.c -o fifo_client_auto
kavya_aluru@Pandu:~/OS_Lab$ ls
builtin.c      fifo_client.c      fifo_server.c      shell_dispatch
export_env     fifo_client_auto  fork_example       shell_dispatch.c
export_env.c   fifo_client_auto.c fork_example.c     state_management
fifo_client    fifo_server       fork_example.c.save state_management.c
kavya_aluru@Pandu:~/OS_Lab$ ./fifo_server
Server started...
Client: Hello Server
Client:
Client: Hello derver
Client: How are you?
Client: Client1 Hello
Client: Client2 Hello
Client: exit
Server stopped.
kavya_aluru@Pandu:~/OS_Lab$ |
```

Report: The program successfully implements client-server communication using named pipes (FIFOs). The client sends messages, and the server processes and responds to them. Multiple clients can communicate through the FIFO, demonstrating inter-process communication and FIFO behavior.

2. Create a POSIX signal handling program that captures SIGINT, SIGTERM, and SIGUSR1. Demonstrate asynchronous event handling and explain the role of signal handlers.

```
#include <stdio.h>
#include <stdlib.h>
#include <signal.h>
#include <unistd.h>
// Signal handler function
void handle_signal(int sig) {
    if (sig == SIGINT) {
        printf("\nSIGINT received: Ctrl+C pressed\n");
    }
    else if (sig == SIGTERM) {
        printf("\nSIGTERM received: Termination request\n");
    }
    else if (sig == SIGUSR1) {
        printf("\nSIGUSR1 received: User signal handled\n");
    }
    fflush(stdout);
}
int main() {
    // Register signal handlers
    signal(SIGINT, handle_signal);
    signal(SIGTERM, handle_signal);
    signal(SIGUSR1, handle_signal);
    printf("Signal handling program started\n");
    printf("Process ID: %d\n", getpid());
    printf("Press Ctrl+C or send signals from another terminal\n");

    while (1) {
        printf("Program running...\n");
        sleep(3);
    }

    return 0;
}
```

```
kavya_aluru@Pandu: ~/OS_Lab$ kill -SIGUSR1 12345
-bash: kill: (12345) - No such process
kavya_aluru@Pandu: ~/OS_Lab$ kill -SIGUSR1 990
kavya_aluru@Pandu: ~/OS_Lab$ kill -SIGTERM 990
kavya_aluru@Pandu: ~/OS_Lab$ kill 990
kavya_aluru@Pandu: ~/OS_Lab$ ps -ef | grep signal_handler
kavya_a+    990      320  0  06:51 pts/0    00:00:00 ./signal_handler
kavya_a+   1054      758  0  06:54 pts/2    00:00:00 grep --color=auto signal_handler
kavya_aluru@Pandu: ~/OS_Lab$ |
```

```
kavya_aluru@Pandu: ~/OS_Lab$ nano signal_handler.c  
kavya_aluru@Pandu:~/OS_Lab$ gcc signal_handler.c -o signal_handler  
kavya_aluru@Pandu:~/OS_Lab$ ./signal_handler  
Signal handling program started  
Process ID: 990  
Press Ctrl+C or send signals from another terminal  
Program running...  
Program running...  
Program running...  
^C  
SIGINT received: Ctrl+C pressed  
Program running...  
Program running...  
Program running...  
Program running...  
Program running...  
Program running...  
Program running...  
Program running...  
Program running...  
Program running...  
Program running...  
Program running...  
Program running...  
Program running...  
  
SIGUSR1 received: User signal handled  
Program running...  
Program running...  
Program running...
```

Report: The program successfully captures SIGINT, SIGTERM, and SIGUSR1 using signal handlers. It demonstrates asynchronous event handling by responding to signals while running, making signal handling simple and reliable.